
Sombras

La importancia de las sombras

En el tema anterior, aprendimos a añadir iluminación a nuestras escenas 3D. Sin embargo, no añadimos luz; en su lugar, simulamos los efectos de la luz sobre los objetos (utilizando el modelo ADS) y modificamos la forma de dibujarlos en consecuencia.

Las limitaciones de este enfoque se hacen evidentes cuando lo utilizamos para iluminar más de un objeto en la misma escena. Considera la escena de la figura 1, que incluye tanto nuestro torus con textura de ladrillo como un plano de tierra (el plano de tierra es la parte superior de un cubo gigante con textura de hierba).

A primera vista, nuestra escena puede parecer razonable. Sin embargo, un examen más detallado revela que falta algo muy importante.

En particular, es imposible discernir la distancia entre el torus y el gran cubo texturizado que se encuentra debajo. ¿El torus flota sobre el cubo o descansa sobre él?



Figura 1: Escena sin sombras.

La razón por la que no podemos responder a esta pregunta se debe a la ausencia de sombras en la escena. Esperamos ver sombras, y nuestro cerebro las utiliza para construir un modelo mental más completo de los objetos que vemos y su ubicación.

Considera la misma escena, mostrada en la figura 2, con sombras incorporadas. Ahora es obvio que el torus descansa sobre el plano del suelo en el ejemplo de la izquierda y flota sobre él en el ejemplo de la derecha.

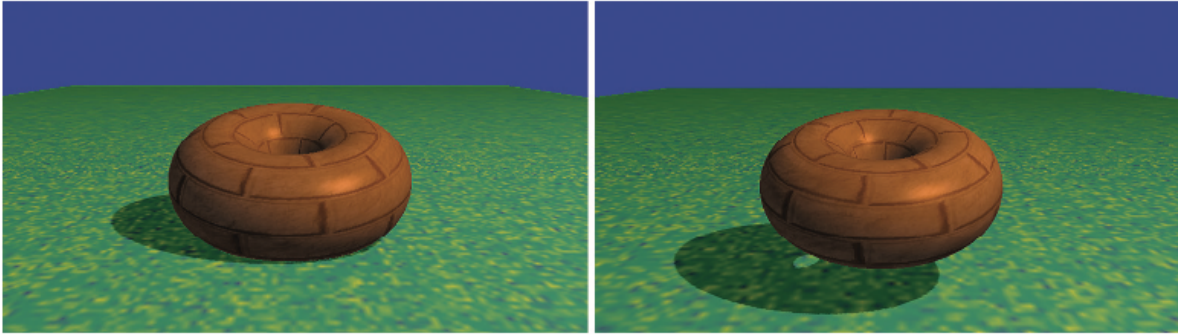


Figura 2: Iluminación con sombras.

Sombras proyectivas

Se han ideado diversos métodos interesantes para añadir sombras a escenas 3D. Un método adecuado para dibujar sombras en un plano del suelo (como nuestra imagen en la figura 1), y relativamente económico desde el punto de vista computacional, se denomina sombras proyectivas.

Dada la posición de una fuente de luz puntual (X_L, Y_L, Z_L) , un objeto a renderizar y un plano sobre el que se proyectará la sombra del objeto, es posible derivar una matriz de transformación que convertirá los puntos (X_W, Y_W, Z_W) del objeto en los puntos de sombra correspondientes (X_S, O, Z_S) del plano. El “polígono de sombra” resultante se dibuja, generalmente como un objeto oscuro fusionado con la textura en el plano del suelo, como se ilustra en la figura 3.

Las ventajas de la proyección de sombras proyectivas son su eficiencia y facilidad de implementación. Sin embargo, solo funciona en un plano plano; el método no puede utilizarse para proyectar sombras en una superficie curva ni en otros objetos. Sigue siendo útil para aplicaciones de alto rendimiento que involucran escenas al aire libre, como en muchos videojuegos.

El desarrollo de matrices de transformación de sombras proyectivas se describe en [BL88¹], [AS14²].

Volúmenes de sombra

Otro método importante, propuesto por Crow en 1977, consiste en identificar el volumen espacial sombreado por un objeto y reducir la intensidad del color de los polígonos dentro de

¹ J. Blinn, “Me and My (Fake) Shadow”, IEEE Computer Graphics and Applications, 8, No. 2 (1988).

² E. Angel and D. Shreiner, Interactive Computer Graphics: A top-down Approach with OpenGL, 7th Ed., Pearson (2014).

la intersección del volumen de sombra con el volumen de vista. La figura 4 muestra un cubo en un volumen de sombra, por lo que este se dibujaría más oscuro.

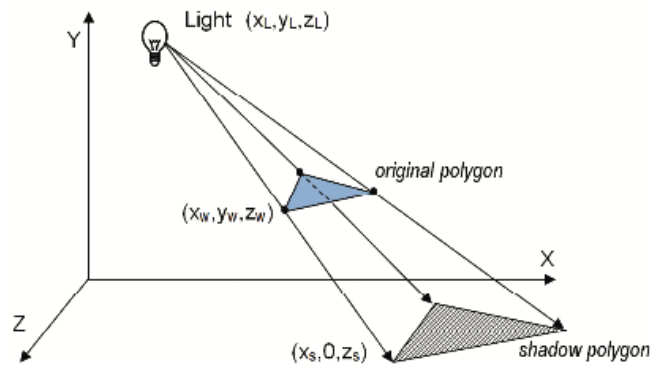


Figura 3: Sombra proyectiva.

Los volúmenes de sombra tienen la ventaja de ser muy precisos y presentan menos artefactos que otros métodos. Sin embargo, determinar el volumen de sombra y calcular si cada polígono se encuentra dentro de él es computacionalmente costoso, incluso en el hardware de GPU moderno.

Se pueden utilizar sombreadores geométricos para generar volúmenes de sombra, y el búfer de plantilla (*stencil buffer*³) se puede utilizar para determinar si un píxel se encuentra dentro del volumen. Algunas tarjetas gráficas incluyen soporte de hardware para optimizar ciertas operaciones de volumen de sombra.

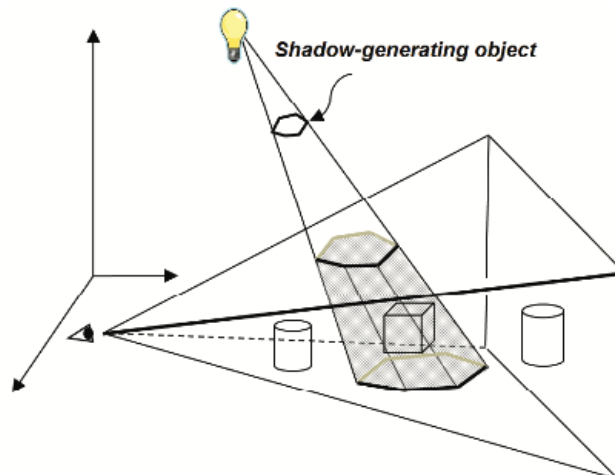


Figura 4: Volumen de sombra.

³ El búfer de plantilla es un tercer búfer, junto con el búfer de color y el búfer z, accesible a través de OpenGL. El búfer de plantilla no se describe en este pequeño curso.

Mapeo de sombras

Uno de los métodos más prácticos y populares para proyectar sombras se denomina mapeo de sombras (*shadow mapping*). Aunque no siempre es tan preciso como los volúmenes de sombra (y suele presentar artefactos molestos), el mapeo de sombras es más fácil de implementar, se puede utilizar en una amplia variedad de situaciones y cuenta con un potente soporte de hardware.

Seríamos negligentes si no aclaráramos el uso del término “más fácil” en el párrafo anterior. Si bien el mapeo de sombras es más sencillo que los volúmenes de sombras (tanto conceptualmente como en la práctica), ¡no es en absoluto “fácil”! Los estudiantes suelen considerar el mapeo de sombras una de las técnicas más difíciles de implementar en un curso de gráficos 3D. Los programas de sombreado son, por naturaleza, difíciles de depurar, y el mapeo de sombras requiere la coordinación perfecta de varios componentes y módulos de sombreado.

Ten en cuenta que la implementación exitosa del mapeo de sombras se verá facilitada en gran medida por el uso generoso de las herramientas de depuración descritas anteriormente en el segundo documento del curso.

El mapeo de sombras se basa en una idea muy simple e ingeniosa: todo lo que no puede ser visto por la luz está en sombra. Es decir, si el objeto #1 impide que la luz llegue al objeto 2, es lo mismo que si la luz no puede “ver” el objeto #2.

La razón por la que esta idea es tan poderosa es que ya contamos con un método para determinar si algo se puede “ver”: el algoritmo de eliminación de superficies ocultas (HSR, *Hidden surface removal*) que utiliza el búfer Z, como se describe en el documento 2. Por lo tanto, una estrategia para encontrar sombras consiste en mover temporalmente la cámara a la ubicación de la luz, aplicar el algoritmo HSR del búfer Z y, a continuación, utilizar la información de profundidad resultante para encontrar sombras.

Renderizar nuestra escena requerirá dos pasadas: una para renderizar la escena desde el punto de vista de la luz (pero sin dibujarla en la pantalla) y una segunda para renderizarla desde el punto de vista de la cámara. El propósito de la primera pasada es generar un búfer Z desde el punto de vista de la luz. Tras completar la primera pasada, debemos conservar el búfer Z y utilizarlo para generar sombras en la segunda. La segunda pasada dibuja la escena.

Nuestra estrategia se está perfeccionando:

- (Pasada 1) Renderizar la escena desde la posición de la luz. El búfer de profundidad contiene, para cada píxel, la distancia entre la luz y el objeto más cercano.
- Copiar el búfer de profundidad a un “búfer de sombras” independiente.
- (Pasada 2) Renderizar la escena con normalidad. Para cada píxel, buscar la posición correspondiente en el búfer de sombras. Si la distancia al punto que se renderiza es mayor que el valor obtenido del búfer de sombras, el objeto que se dibuja en este píxel está más lejos de la luz que el objeto más cercano a ella y, por lo tanto, este píxel está en sombra.

Cuando un píxel está en sombra, debemos oscurecerlo. Una forma sencilla y eficaz de hacerlo es renderizar solo su iluminación ambiental, ignorando sus componentes difusos y especulares.

El método descrito anteriormente se suele denominar “búfer de sombras”. El término “mapeo de sombras” surge cuando, en el segundo paso, copiamos el búfer de profundidad en una textura. Cuando un objeto de textura se utiliza de esta manera, lo denominaremos textura de sombra, y OpenGL admite texturas de sombra mediante el tipo `sampler2DShadow` (que se describe más adelante).

Esto nos permite aprovechar la potencia del soporte de hardware para unidades de textura y variables de sampler (es decir, “mapeo de texturas”) en el sombreador de fragmentos para realizar rápidamente la búsqueda de profundidad en el paso 2. Nuestra estrategia revisada ahora es:

- (Pasada 1) Igual que antes.
- Copiar el búfer de profundidad en una textura.
- (Pasada 2) Igual que antes, excepto que el búfer de sombra ahora es una textura de sombra.

Implementemos ahora estos pasos.

Mapeo de Sombras (Pasada uno) – Dibujar objetos a partir de la posición de la luz

En el paso uno, primero movemos la cámara a la posición de la luz y luego renderizamos la escena. Nuestro objetivo no es dibujar la escena en la pantalla, sino completar el proceso de renderizado lo suficiente para que el búfer de profundidad se llene correctamente. Por lo tanto, no será necesario generar colores para los píxeles, por lo que nuestra primera pasada utilizará un sombreador de vértices, pero el sombreador de fragmentos no hará nada.

Por supuesto, mover la cámara implica construir una matriz de vista adecuada. Dependiendo del contenido de la escena, tendremos que decidir la *dirección* adecuada para verla desde la

luz. Normalmente, querríamos que esta dirección sea hacia la región que finalmente se renderiza en la pasada 2.

Esto suele ser específico de la aplicación; en nuestras escenas, generalmente apuntaremos la cámara desde la luz hacia el origen.

En la primera pasada, es necesario gestionar varios detalles importantes:

- Configurar el búfer y la textura de la sombra.
- Desactivar la salida de color.
- Construir una matriz de vista desde la luz hacia los objetos a la vista.
- Habilitar el programa de sombreado GLSL de la primera pasada, que contiene únicamente el sombreador de vértices simple que se muestra en la figura 5 y que espera recibir una matriz MVP. En este caso, la matriz MVP incluirá la matriz de modelo M del objeto, la matriz de vista calculada en el paso anterior (que actúa como matriz de vista V) y la matriz de perspectiva P. A esta matriz MVP la llamamos "shadowMVP" porque se basa en el punto de vista de la luz y no en el de la cámara. Dado que la vista desde la luz no se muestra realmente, el sombreador de fragmentos del programa de la primera pasada no realiza ninguna acción.
- Para cada objeto, crear la matriz shadowMVP y llamar a `glDrawArrays()`. No es necesario incluir texturas ni iluminación en la primera pasada, ya que los objetos no se renderizan en pantalla.

```
#version 430           // vertex shader
layout (location=0) in vec3 vertPos;
uniform mat4 shadowMVP;

void main(void)
{   gl_Position = shadowMVP * vec4(vertPos,1.0);
}
```

```
#version 430           // fragment shader
void main(void) { }
```

Figura 5: Mapeo de sombras, pasada 1, sombreadores de vértices y fragmentos.

Mapeo de sombras (paso intermedio) – Copia del búfer Z a una textura

OpenGL ofrece dos métodos para introducir los datos de profundidad del búfer Z en una unidad de textura. El primer método consiste en generar una textura de sombra vacía y luego usar el comando `glCopyTexImage2D()` para copiar el búfer de profundidad activo en la textura de sombra.

El segundo método consiste en crear un "framebuffer personalizado" en la primera pasada (en lugar de usar el búfer Z predeterminado) y asociarle la textura de sombra mediante el comando `glFramebufferTexture()`. Este comando se introdujo en OpenGL en la versión 3.0 para mejorar la compatibilidad con el mapeo de sombras. Al usar este enfoque, no es necesario "copiar" el búfer Z en una textura, ya que el búfer ya tiene una textura asociada, por lo que OpenGL introduce automáticamente la información de profundidad en la textura. Este es el método que usaremos en nuestra implementación.

Mapeo de sombras (Pasada dos) – Renderizado de la escena con sombras

La pasada dos se asemejará en gran medida a lo que vimos en el documento anterior. Es decir, aquí renderizamos la escena completa y todos los elementos que la componen, junto con la iluminación, los materiales y las texturas que adornan los objetos. También necesitamos agregar el código necesario para determinar, para cada píxel, si está o no en sombra.

Una característica importante del paso dos es que utiliza dos matrices MVP. Una es la matriz MVP estándar que transforma las coordenadas de los objetos en coordenadas de pantalla (como se vio en la mayoría de nuestros ejemplos anteriores). La otra es la matriz `shadowMVP`, generada en la primera pasada para renderizar desde el punto de vista de la luz. Esta se usará ahora en la segunda pasada para consultar la información de profundidad de la textura de la sombra.

En la segunda pasada, surge una complicación al consultar píxeles en un mapa de textura. La cámara OpenGL utiliza un espacio de coordenadas $[-1..+1]$, mientras que los mapas de textura utilizan un espacio $[0..1]$. Una solución común es crear una transformación de matriz adicional, comúnmente llamada *B*, que convierte (o "sesga", de ahí su nombre) del espacio de la cámara al espacio de textura. Obtener *B* es bastante sencillo: una escala a la mitad seguida de una traslación a la mitad. La matriz *B* es la siguiente:

$$B = \begin{bmatrix} 0.5 & 0 & 0 & 0.5 \\ 0 & 0.5 & 0 & 0.5 \\ 0 & 0 & 0.5 & 0.5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

B se concatena con la matriz `shadowMVP` para su uso en la segunda pasada, como se indica a continuación:

$$\text{shadowMVP2} = [B] [\text{shadowMVP}_{(\text{pasada1})}]$$

Suponiendo que utilizamos el método mediante el cual se adjunta una textura de sombra a nuestro framebuffer personalizado, OpenGL proporciona herramientas relativamente sencillas para determinar si cada píxel está en sombra al dibujar los objetos. A continuación, se presenta un resumen de los detalles gestionados en la segunda pasada:

- Construir la matriz de transformación "B" para la conversión del espacio de luz al espacio de textura (de hecho, esto se realiza de forma más adecuada en `init()`).
- Habilitar la textura de sombra para la búsqueda.
- Habilitar la salida de color.
- Habilitar el programa de renderizado GLSL de la segunda pasada, que contiene sombreadores de vértices y de fragmentos.
- Construir la matriz MVP para el objeto que se está dibujando en función de la posición de la cámara (como de costumbre).
- Construir la matriz `shadowMVP2` (incorporando la matriz B, como se describió anteriormente). Los sombreadores la necesitarán para buscar las coordenadas de los píxeles en la textura de la sombra.
- Enviar las transformaciones de la matriz a las variables uniformes del sombreador.
- Habilitar los búferes que contienen vértices, vectores normales y coordenadas de textura (si se utilizan), como de costumbre.
- Llamar a `glDrawArrays()`.

Además de sus funciones de renderizado, los sombreadores de vértices y fragmentos tienen otras tareas:

- El sombreador de vértices convierte las posiciones de los vértices del espacio de la cámara al espacio de la luz y envía las coordenadas resultantes al sombreador de fragmentos en un atributo de vértice para su interpolación. Esto permite recuperar los valores correctos de la textura de sombra.
- El sombreador de fragmentos llama a la función `textureProj()`, que devuelve un 0 o un 1 indicando si el píxel está en sombra (este mecanismo se explica más adelante). Si está en sombra, el sombreador genera un píxel más oscuro al omitir sus contribuciones difusas y especulares.

El mapeo de sombras es una tarea tan común que GLSL proporciona un tipo especial de variable de muestreador llamada `sampler2DShadow` (como se mencionó anteriormente) que se puede asociar a una textura de sombra en la aplicación C++/OpenGL. La función `textureProj()` se utiliza para buscar valores de una textura de sombra y es similar a la función `texture()` que vimos en el tema 5, excepto que utiliza un `vec3` para indexar la textura en lugar del `vec2` habitual. Dado que la coordenada de un píxel es un `vec4`, es necesario proyectarla en el espacio de textura 2D para buscar el valor de profundidad en el mapa de textura de

sombra. Como veremos a continuación, `textureProj()` se encarga de todo esto automáticamente.

El resto del código del sombreador de vértices y fragmentos implementa el sombreado Blinn-Phong. Estos sombreadores se muestran en las figuras 6 y 7, con el código añadido para el mapeo de sombras resaltado.

Examinemos más detenidamente cómo usamos OpenGL para realizar la comparación de profundidad entre el píxel renderizado y el valor en la textura de sombra. Comenzamos en el sombreador de vértices con las coordenadas de los vértices en el espacio modelo, que multiplicamos por `shadowMVP2` para generar las coordenadas de la textura de sombra, correspondientes a las coordenadas de los vértices proyectadas en el espacio de luz, previamente generadas desde el punto de vista de la luz.

Las coordenadas interpoladas (3D) del espacio de luz (x, y, z) se utilizan en el sombreador de fragmentos de la siguiente manera. El componente z representa la distancia de la luz al píxel. Los componentes (x, y) se utilizan para recuperar la información de profundidad almacenada en la textura de sombra (2D). Este valor obtenido (la distancia al objeto más cercano a la luz) se compara con z. Esta comparación produce un resultado "binario" que indica si el píxel que estamos renderizando está más alejado de la luz que el objeto más cercano (es decir, si el píxel está en sombra).

```
#version 430
layout (location=0) in vec3 vertPos;
layout (location=1) in vec3 vertNormal;

out vec3 varyingNormal, varyingLightDir, varyingVertPos, varyingHalfVec;
out vec4 shadow_coord;

struct PositionalLight { vec4 ambient, diffuse, specular; vec3 position; };
struct Material { vec4 ambient, diffuse, specular; float shininess; };
uniform vec4 globalAmbient;
uniform PositionalLight light;    // light's position is assumed to be in eye space
uniform Material material;
uniform mat4 m_matrix;
uniform mat4 v_matrix;
uniform mat4 p_matrix;
uniform mat4 norm_matrix;
uniform mat4 shadowMVP2;
layout (binding=0) uniform sampler2DShadow shTex;

void main(void)
{
    varyingVertPos = (m_matrix * vec4(vertPos,1.0)).xyz;
    varyingLightDir = light.position - varyingVertPos;
    varyingNormal = (norm_matrix * vec4(vertNormal,1.0)).xyz;
    varyingHalfVec = (varyingLightDir - varyingVertPos).xyz;
    shadow_coord = shadowMVP2 * vec4(vertPos,1.0);
    gl_Position = p_matrix * v_matrix * m_matrix * vec4(vertPos,1.0);
}
```

Figura 6: Sombras, pasada 2 del sombreador de vértices.

```

#version 430
in vec3 varyingNormal, varyingLightDir, varyingVertPos, varyingHalfVec;
in vec4 shadow_coord;
out vec4 fragColor;
// same structs and uniforms as in the vertex shader

void main(void)
{
    vec3 L = normalize(varyingLightDir);
    vec3 N = normalize(varyingNormal);
    vec3 V = normalize(-v.matrix[3].xyz - varyingVertPos);
    vec3 H = normalize(varyingHalfVec);

    float inShadow = textureProj(shTex, shadow_coord);

    fragColor = globalAmbient * material.ambient + light.ambient * material.ambient;
    if (notInShadow == 1.0)
    {
        fragColor += light.diffuse * material.diffuse * max(dot(L,N),0.0)
            + light.specular * material.specular
            * pow(max(dot(H,N),0.0),material.shininess*3.0);
    }
}

```

Figura 7: Sombras, pasada 2 del sombreador de fragmentos.

Si en OpenGL usamos `glFramebufferTexture()` como se describió anteriormente y habilitamos las pruebas de profundidad, usar `sampler2DShadow` y `textureProj()`, como se muestra en el sombreador de fragmentos (Figura 7), hará exactamente lo que necesitamos. Es decir, `textureProj()` generará 0.0 o 1.0 según la comparación de profundidad. Con base en este valor, podemos omitir en el sombreador de fragmentos las contribuciones difusa y especular cuando el píxel está más alejado de la luz que el objeto más cercano, creando así la sombra cuando sea necesario. La figura 8 muestra una descripción general.

Ahora estamos listos para compilar nuestra aplicación C++/OpenGL para trabajar con los sombreadores descritos anteriormente.

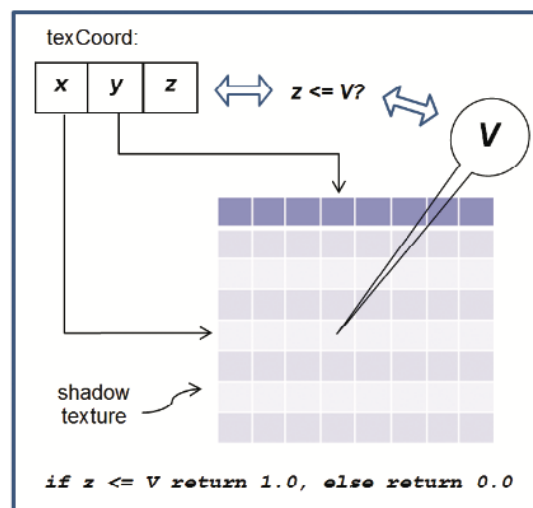


Figura 8: Comparación automática de profundidad.

Un ejemplo de mapeado de sombras

Considera la escena de la figura 9, que incluye un torus y una pirámide. Se ha colocado una luz de posición a la izquierda (observa los reflejos especulares).

La pirámide *debería* proyectar una sombra sobre el torus. Para aclarar el desarrollo del ejemplo, nuestro primer paso será renderizar la primera pasada en la pantalla para asegurarnos de que funcione correctamente.

Para ello, añadiremos temporalmente un sombreador de fragmentos simple (no se incluirá en la versión final) a la primera pasada que solo genera un color constante (por ejemplo, rojo); por ejemplo:

```
#version 430
out vec4 fragColor;
void main(void)
{ fragColor = vec4(1.0, 0.0, 0.0, 0.0);
}
```

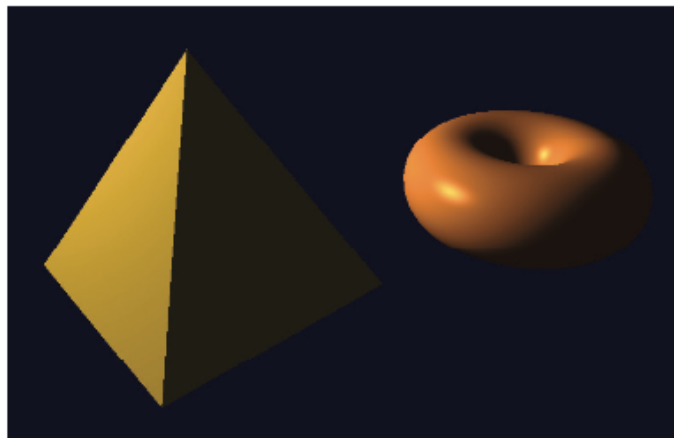


Figura 9: Escena iluminada sin sombras.

Supongamos que el origen de la escena anterior se sitúa en el centro de la figura, entre la pirámide y el torus. En la primera pasada, colocamos la cámara en la posición de la luz (a la izquierda en la figura 10) y la apuntamos hacia (0,0,0). Si dibujamos los objetos en rojo, se obtiene el resultado que se muestra a la derecha en la figura 10.

Observa el torus cerca de la parte superior; desde este punto de vista, está parcialmente detrás de la pirámide.

El código completo de C++/OpenGL de dos pasadas con iluminación y mapeo de sombras se muestra en el Programa 8.1.

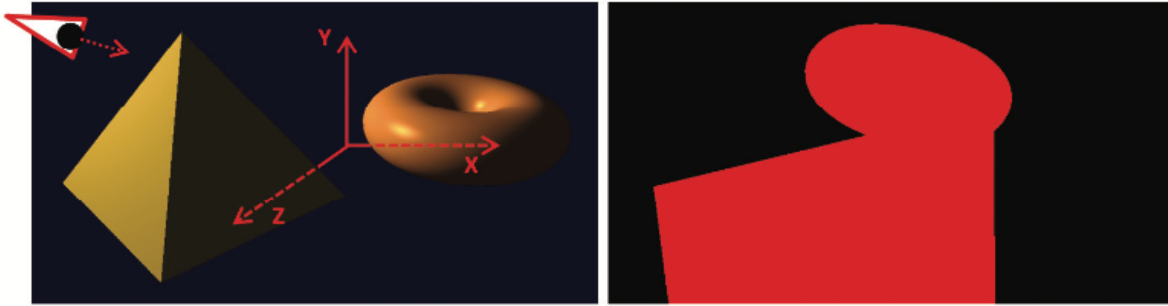


Figura 10: Primera pasada: Escena (izquierda) desde el punto de vista de la luz (derecha).

Programa 8.1 Mapeo de sombras

Revisar el código adjunto al documento.

El programa 8.1 muestra las partes relevantes de la aplicación C++/OpenGL que interactúan con los sombreadores de pasada uno y pasada dos, detallados previamente.

No se muestran los módulos habituales para leer y compilar los sombreadores, construir los modelos y sus búferes relacionados, instalar las características ADS de la luz posicional en los sombreadores y realizar los cálculos de perspectiva y matriz de observación. Estos no han cambiado con respecto a los ejemplos anteriores.

Artefactos de mapeo de sombras

Aunque hemos implementado todos los requisitos básicos para añadir sombras a nuestra escena, la ejecución del programa 8.1 produce resultados mixtos, como se muestra en la figura 11.

La buena noticia es que nuestra pirámide ahora proyecta una sombra sobre el torus. Desafortunadamente, este éxito viene acompañado de un artefacto grave. Hay líneas onduladas que cubren muchas de las superficies de la escena.

Esta es una consecuencia común del mapeo de sombras y se denomina acné de sombra (*shadow acne*) o auto-sombreado erróneo (*erroneous self-shadowing*).

El acné de sombra se debe a errores de redondeo durante las pruebas de profundidad. Las coordenadas de textura calculadas al consultar la información de profundidad en una textura de sombra a menudo no coinciden exactamente con las coordenadas reales.

Por lo tanto, la búsqueda puede devolver la profundidad de un píxel vecino, en lugar de la que se está renderizando. Si la distancia al píxel vecino es mayor, nuestro píxel parecerá estar en sombra aunque no lo esté.

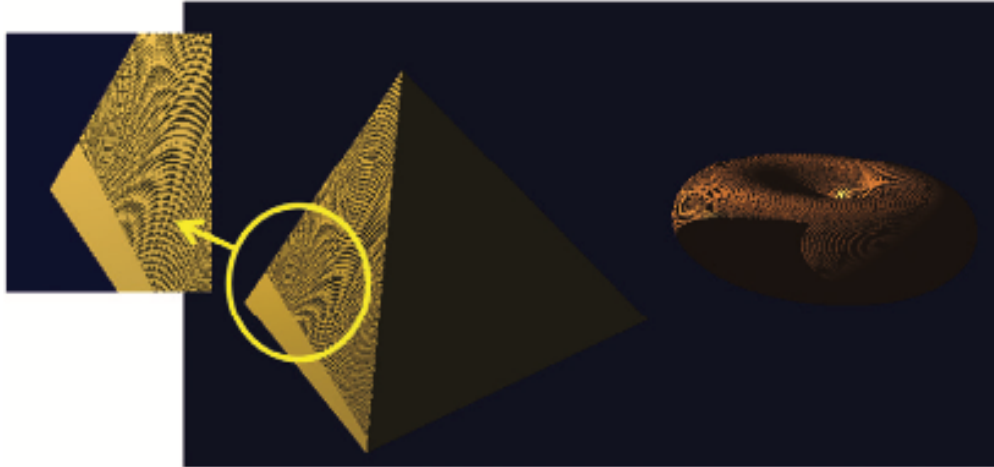


Figura 11: “Acné” de sombra.

El acné de sombra también puede deberse a diferencias de precisión entre el mapa de textura y el cálculo de profundidad. Esto también puede provocar errores de redondeo y, por consiguiente, una evaluación incorrecta de si un píxel está o no en sombra.

Afortunadamente, solucionar el acné de sombra es bastante sencillo. Dado que el acné de sombra suele aparecer en superficies que no están en sombra, un truco sencillo consiste en acercar ligeramente cada píxel a la luz durante la primera pasada y luego devolverlos a sus posiciones normales en la segunda.

Esto suele ser suficiente para compensar cualquier tipo de error de redondeo. Una forma sencilla es llamar a `glPolygonOffset()` en la función `display()`, como se muestra en la figura 12 (resaltada).

Añadir estas pocas líneas de código a nuestra función `display()` mejora considerablemente el resultado de nuestro programa, como se muestra en la figura 13. Observa también que, una vez eliminados los artefactos, ahora podemos ver que el círculo interior del torus muestra una pequeña sombra correctamente proyectada sobre sí mismo.

Aunque corregir el acné de sombra es fácil, a veces la reparación provoca nuevos artefactos. El truco de mover el objeto antes de la primera pasada puede provocar que aparezca un hueco dentro de la sombra de un objeto. Un ejemplo de esto se muestra en la figura 14.

```

void display(GLFWwindow* window, double currentTime) {
    // clearing the depth and color buffers as before goes here
    ...
    // setting up matrices for camera and light points of view as before goes here
    ...
    glBindFramebuffer(GL_FRAMEBUFFER, shadowBuffer);
    glFramebufferTexture(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, shadowTex, 0);

    glDrawBuffer(GL_NONE);
    glEnable(GL_DEPTH_TEST);

    // for reducing shadow artifacts
    glEnable(GL_POLYGON_OFFSET_FILL);
    glPolygonOffset(2.0f, 4.0f);

    passOne();

    glDisable(GL_POLYGON_OFFSET_FILL);

    glBindFramebuffer(GL_FRAMEBUFFER, 0);
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, shadowTex);
    glDrawBuffer(GL_FRONT);

    passTwo();
}

```

Figura 12: Combatiendo el acné de sombra.

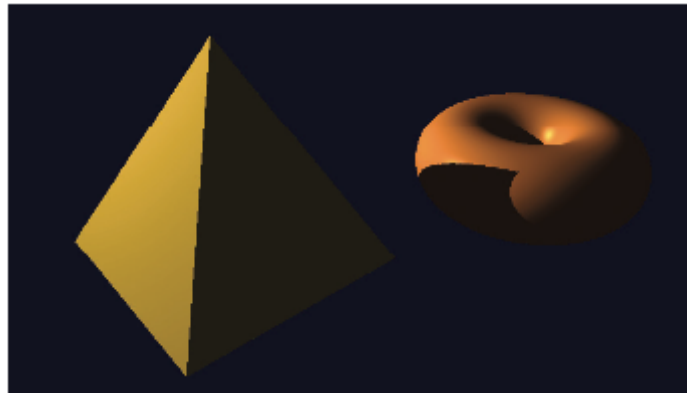


Figura 13: Escena renderizada con sombras.

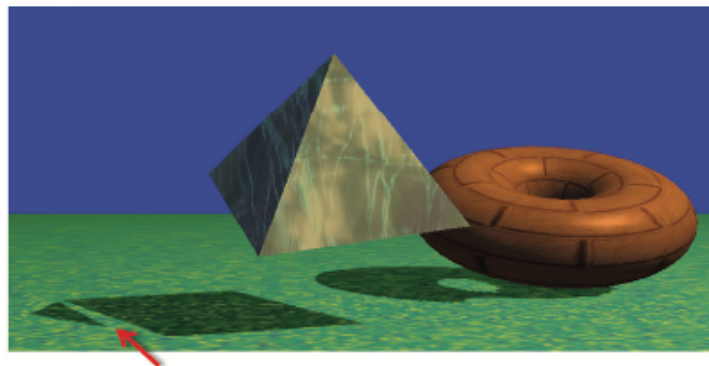


Figura 14: "Peter Panning".

Este artefacto se suele llamar “Peter Panning”, ya que a veces provoca que la sombra de un objeto en reposo se separe inapropiadamente de su base (haciendo que partes de la sombra del objeto se desprendan del resto de la sombra, recordando al personaje Peter Pan de J. M. Barrie).

Para corregir este artefacto, es necesario ajustar los parámetros `glPolygonOffset()`. Si son demasiado pequeños, puede aparecer acné de sombra; si son demasiado grandes, se produce Peter Panning.

Existen muchos otros artefactos que pueden producirse durante el mapeo de sombras. Por ejemplo, las sombras pueden *repetirse* debido a que la región de la escena que se renderiza en la primera pasada (en el búfer de sombras) es diferente de la región de la escena renderizada en la segunda pasada (desde diferentes puntos de vista).

Debido a esta diferencia, las partes de la escena renderizadas en la segunda pasada que queden fuera de la región renderizada en la primera pasada intentarán acceder a la textura de la sombra utilizando coordenadas de textura fuera del rango `[0..1]`. Recuerda que el comportamiento predeterminado en este caso es `GL_REPEAT`, lo que puede resultar en sombras duplicadas incorrectamente.

Una posible solución es agregar las siguientes líneas de código a `setupShadowBuffers()` para establecer el modo de ajuste de textura en “fijar al borde” (*clamp to edge*):

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
```

Esto hace que los valores fuera del borde de una textura se fijen al valor en el borde (en lugar de repetirse). Ten en cuenta que este enfoque puede generar sus propios artefactos; es decir, cuando existe una sombra en el borde de la textura de sombra, fijarla al borde puede producir una “barra de sombra” (*shadow bar*) que se extiende hasta el borde de la escena.

Otro error común son los bordes de sombra irregulares (*jagged shadow edges*). Esto puede ocurrir cuando la sombra proyectada es significativamente mayor de lo que el búfer de sombras puede representar con precisión. Esto suele depender de la ubicación de los objetos y la(s) luz(es) en la escena. En particular, suele ocurrir cuando la fuente de luz está relativamente lejos de los objetos involucrados. La figura 15 muestra un ejemplo.

Eliminar los bordes de sombra irregulares no es tan sencillo como en el caso de los artefactos anteriores. Una técnica consiste en acercar la posición de la luz a la escena durante la primera pasada y luego devolverla a la posición correcta en la segunda. Otro enfoque que suele ser eficaz es emplear uno de los métodos de “sombra suave” (*soft shadow*) que analizaremos a continuación.

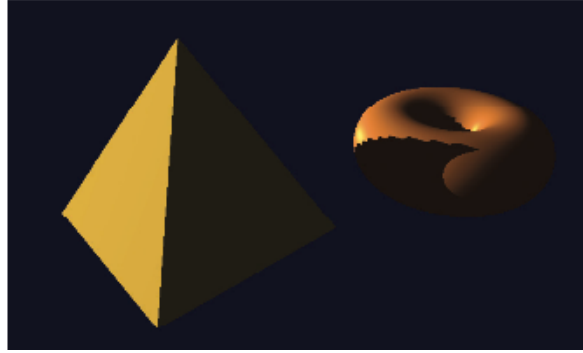


Figura 15: Bordes de sombra irregulares.

Sombras suaves

Los métodos presentados hasta ahora se limitan a producir sombras duras. Estas son sombras con bordes definidos. Sin embargo, la mayoría de las sombras que se producen en el mundo real son sombras suaves. Es decir, sus bordes están difuminados en diversos grados. En esta sección, exploraremos la apariencia de las sombras suaves tal como ocurren en el mundo real y luego describiremos un algoritmo comúnmente usado para simularlas en OpenGL.

Sombras suaves en el mundo real

Existen muchas causas y tipos de sombras suaves. Una causa común de sombras suaves en la naturaleza es que las fuentes de luz del mundo real rara vez son puntuales; con mayor frecuencia, ocupan un área.

Otra causa es la acumulación de imperfecciones en materiales y superficies, y el papel que los propios objetos desempeñan en la generación de luz ambiental a través de sus propias propiedades reflectantes.

La figura 16 muestra una fotografía de un objeto que proyecta una sombra suave sobre una mesa. Cabe destacar que no se trata de una representación 3D por computadora, sino de una fotografía real de un objeto, tomada en la casa de uno de los autores.

Hay dos aspectos a destacar sobre la sombra en la figura 16:

- La sombra es “más suave” cuanto más lejos está del objeto y “más dura” cuanto más cerca está del objeto. Esto se aprecia al comparar la sombra cerca de las patas del objeto con la porción más ancha de la sombra en la región derecha de la imagen.
- La sombra se ve ligeramente más oscura cuanto más cerca está del objeto.



Figura 16: Ejemplo real de sombra suave.

La propia dimensionalidad de la fuente de luz puede generar sombras suaves. Como se muestra en la figura 17, las distintas regiones de la fuente de luz proyectan sombras ligeramente diferentes. Las áreas más claras en los bordes de la sombra, donde solo una parte de la luz es bloqueada por el objeto, se denominan conjuntamente penumbra.

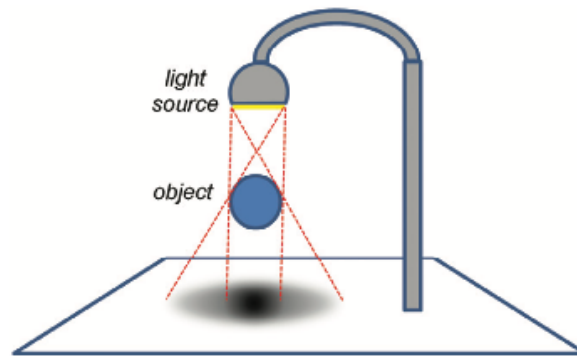


Figura 17: Efecto de penumbra de sombra suave.

Generación de sombras suaves: Filtrado de porcentaje de cierre (PCF, *percentage closer filtering*)

Existen varias maneras de simular el efecto de penumbra para generar sombras suaves en software. Una de las más sencillas y comunes se denomina filtrado de porcentaje de cierre (PCF). En PCF (propuesto en 1987 por Reeves et al.), muestreamos la textura de la sombra en varias ubicaciones circundantes para estimar el porcentaje de ubicaciones cercanas que están en sombra.

Dependiendo de cuántas ubicaciones cercanas estén en sombra, aumentamos o disminuimos el grado de contribución de la iluminación del píxel que se está renderizando. Todo el cálculo se puede realizar en el sombreador de fragmentos, y ese es el único lugar donde tenemos que modificar el código. PCF también se puede utilizar para reducir los artefactos de líneas irregulares.

Antes de estudiar el algoritmo PCF propiamente dicho, veamos un ejemplo simple similar y motivador para ilustrar el objetivo de PCF. Consideremos el conjunto de fragmentos de salida (píxeles) que se muestra en la figura 18, cuyos colores son calculados por el sombreador de fragmentos.

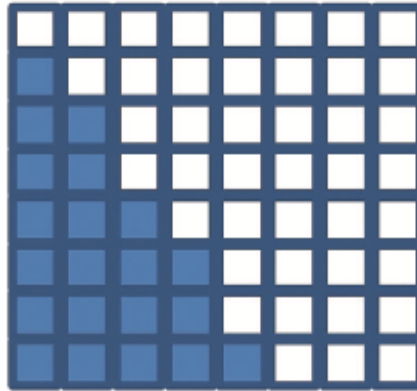


Figura 18: Renderizado de sombras duras.

Supongamos que los píxeles oscurecidos están en sombra, como se calcula mediante el mapeo de sombras. En lugar de simplemente renderizar los píxeles como se muestra (es decir, con o sin los componentes difuso y especular incluidos), supongamos que tuviéramos acceso a la información de los píxeles vecinos para poder ver cuántos están en sombra. Por ejemplo, considera el píxel resaltado en amarillo en la figura 19, que, según la figura 18, no está en sombra.

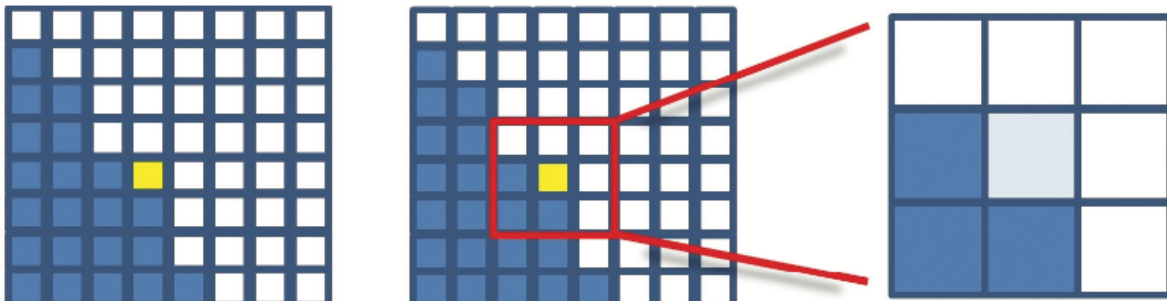


Figura 19: Muestreo PCF para un píxel específico.

En la vecindad de nueve píxeles del píxel resaltado, tres píxeles están en sombra y seis no. Por lo tanto, el color del píxel renderizado podría calcularse como la suma de la contribución ambiental en ese píxel, más seis novenos de las contribuciones difusa y especular, lo que resulta en un píxel bastante (pero no completamente) iluminado.

Repetir este proceso a lo largo de la cuadrícula produciría colores de píxeles aproximadamente como los que se muestran en la figura 20. Ten en cuenta que para los

píxeles cuyas vecindades están completamente en (o fuera de) sombra, el color resultante es el mismo que para el mapeo de sombras estándar.

A diferencia del ejemplo que se acaba de mostrar, las implementaciones de PCF no muestrean todos los píxeles dentro de una vecindad específica del píxel que se está renderizando. Hay dos razones para esto: (a) nos gustaría realizar este cálculo en el sombreador de fragmentos, pero este no tiene acceso a otros píxeles; y (b) obtener un efecto de penumbra suficientemente amplio (por ejemplo, de diez a veinte píxeles de ancho) requeriría muestrear cientos de píxeles cercanos por cada píxel que se está renderizando.

PCF aborda estos dos problemas de la siguiente manera. Primero, en lugar de intentar acceder a los píxeles cercanos, muestreamos los téxels cercanos en el mapa de sombras. El sombreador de fragmentos puede hacer esto porque, aunque no tiene acceso a los valores de los píxeles cercanos, sí tiene acceso a todo el mapa de sombras. Segundo, para lograr un efecto de penumbra suficientemente amplio, se muestrea un número moderado de téxels del mapa de sombras cercanos, cada uno a una distancia moderada del téxel correspondiente al píxel que se está renderizando.

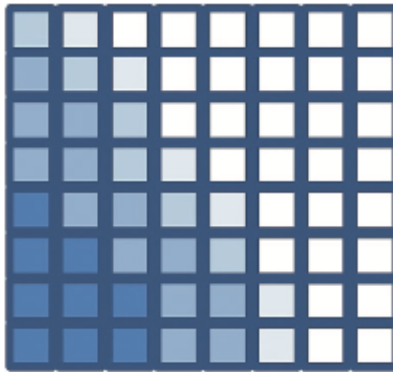


Figura 20: Renderizado de sombras suaves.

La anchura de la penumbra y el número de puntos muestreados se pueden ajustar en función de la escena y los requisitos de rendimiento. Por ejemplo, la imagen que se muestra en la figura 21 se generó mediante PCF, y el brillo de cada píxel se determinó muestreando 64 téxels del mapa de sombras cercanos a diferentes distancias del téxel del píxel.

La precisión o suavidad de nuestras sombras suaves depende del número de téxels cercanos muestreados. Por lo tanto, existe un equilibrio entre rendimiento y calidad: cuantos más puntos se muestrean, mejores son los resultados, pero mayor es la sobrecarga computacional. Dependiendo de la complejidad de la escena y la velocidad de fotogramas requerida para una aplicación determinada, suele existir un límite práctico correspondiente a la calidad que se puede lograr. Muestrear 64 puntos por píxel, como en la figura 21, suele ser poco práctico.

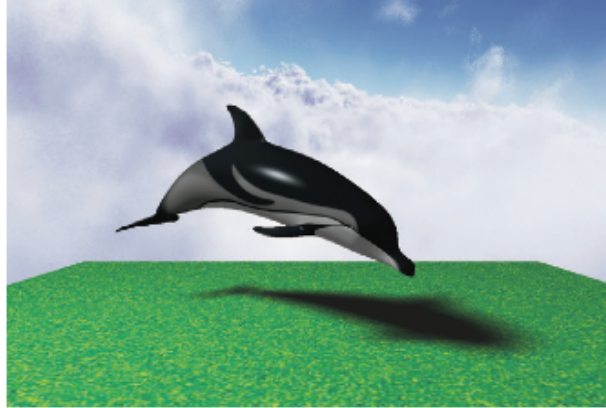


Figura 21: Renderizado de sombras suaves: 64 muestras por píxel.

Un algoritmo comúnmente utilizado para implementar PCF consiste en muestrear cuatro texeles cercanos por píxel, seleccionando las muestras a distancias específicas del texel correspondiente al píxel. A medida que procesamos cada píxel, modificamos las compensaciones utilizadas para determinar qué cuatro texeles se muestrean. Modificar las compensaciones de forma escalonada se denomina a veces *dithering* y tiene como objetivo que el límite de la sombra suave parezca menos “bloqueado” de lo que sería normalmente dado el pequeño número de puntos de muestra.

Un enfoque común consiste en asumir uno de cuatro patrones de desplazamiento diferentes. Podemos elegir el patrón que usaremos para un píxel determinado calculando su `glFragCoord mod 2`. Recordemos que `glFragCoord` es de tipo `vec2` y contiene las coordenadas x e y de la ubicación del píxel; el resultado del cálculo del `mod` es uno de cuatro valores: $(0,0)$, $(0,1)$, $(1,0)$ o $(1,1)$. Utilizamos este resultado para seleccionar uno de nuestros cuatro patrones de desplazamiento diferentes en el espacio de texeles (es decir, en el mapa de sombras).

Los patrones de desplazamiento se especifican típicamente en las direcciones x e y con diferentes combinaciones de -1.5 , -0.5 , $+0.5$ y $+1.5$ (que también se pueden escalar según se desee).

Más específicamente, los cuatro patrones de desplazamiento habituales para cada caso resultantes del cálculo de `glFragCoord mod 2` son:

case (0,0)	case (0,1)	case (1,0)	case (1,1)
<i>sample points:</i>	<i>sample points:</i>	<i>sample points:</i>	<i>sample points:</i>
$(s_x - 1.5, s_y + 1.5)$	$(s_x - 1.5, s_y + 0.5)$	$(s_x - 0.5, s_y + 1.5)$	$(s_x - 0.5, s_y + 0.5)$
$(s_x - 1.5, s_y - 0.5)$	$(s_x - 1.5, s_y - 1.5)$	$(s_x - 0.5, s_y - 0.5)$	$(s_x - 0.5, s_y - 1.5)$
$(s_x + 0.5, s_y + 1.5)$	$(s_x + 0.5, s_y + 0.5)$	$(s_x + 1.5, s_y + 1.5)$	$(s_x + 1.5, s_y + 0.5)$
$(s_x + 0.5, s_y - 0.5)$	$(s_x + 0.5, s_y - 1.5)$	$(s_x + 1.5, s_y - 0.5)$	$(s_x + 1.5, s_y - 1.5)$

S_x y S_y se refieren a la ubicación (S_x, S_y) en el mapa de sombras correspondiente al píxel que se está renderizando, identificado como `shadow_coord` en los ejemplos de código de este documento. Estos cuatro patrones de desplazamiento se ilustran en la figura 22, donde cada caso se muestra en un color diferente. En cada caso, el texel correspondiente al píxel que se está renderizando se encuentra en el origen del grafo correspondiente. Notemos que, al mostrarse juntos en la figura 23, el escalonamiento/tramado de los desplazamientos es evidente.

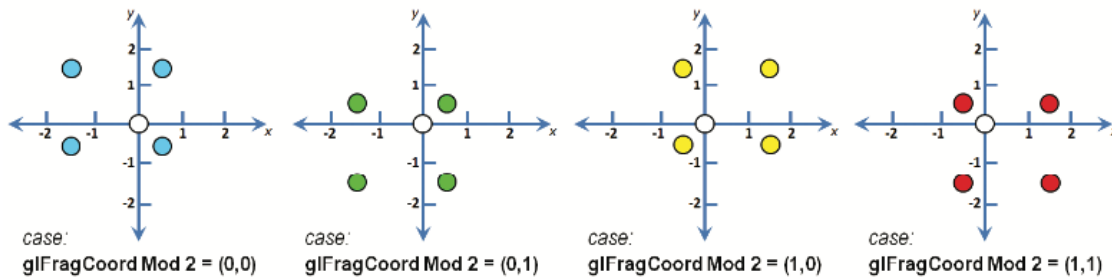


Figura 22: Casos de muestreo PCF de cuatro píxeles con tramado.

Repasemos el cálculo completo para un píxel en particular. Supongamos que el píxel que se renderiza se encuentra en `glFragCoord = (48.13)`. Comenzamos determinando los cuatro puntos de muestra del mapa de sombras para el píxel. Para ello, calcularíamos `vec2(48.13) mod 2`, que es igual a $(0,1)$.

A partir de esto, elegiríamos los desplazamientos mostrados para el caso $(0,1)$, que se muestran en verde en la figura 22, y los puntos específicos que se muestrearán en el mapa de sombras (suponiendo que no se ha especificado ninguna escala de desplazamientos) serían:

- $(\text{shadow_coord.x}-1.5, \text{shadow_coord.y}+0.5)$
- $(\text{shadow_coord.x}-1.5, \text{shadow_coord.y}-1.5)$
- $(\text{shadow_coord.x}+0.5, \text{shadow_coord.y}+0.5)$
- $(\text{shadow_coord.x}+0.5, \text{shadow_coord.y}-1.5)$

(Recuerda que `shadow_coord` es la ubicación del texel en el mapa de sombras correspondiente al píxel que se está renderizando; se muestra como un círculo blanco en las figuras 22 y 23).

A continuación, llamamos a la función `textureProj()` en cada uno de estos cuatro puntos, que en cada caso devuelve 0.0 o 1.0 según si el punto muestreado está o no en sombra. Sumamos los cuatro resultados y los dividimos entre 4.0 para determinar el porcentaje de puntos muestreados que están en sombra. Este porcentaje se utiliza como multiplicador para determinar la cantidad de iluminación difusa y especular que se aplicará al renderizar el píxel actual.

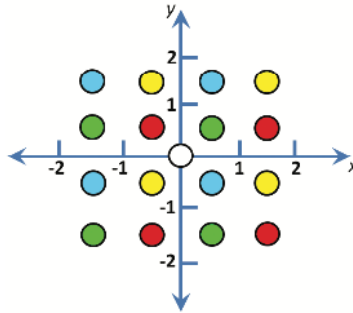


Figura 23: Muestreo PCF de cuatro píxeles con tramado (cuatro casos mostrados juntos).

A pesar del pequeño tamaño de muestreo (solo cuatro muestras por píxel), este enfoque de tramado a menudo puede producir sombras suaves sorprendentemente buenas. La figura 24 se generó utilizando PCF con tramado de cuatro puntos. Si bien no es tan buena como la versión muestreada de 64 puntos mostrada anteriormente en la figura 21, se renderiza considerablemente más rápido.

En la siguiente sección, desarrollamos el sombreador de fragmentos GLSL que produjo tanto esta sombra suave PCF con tramado de cuatro muestras como la sombra suave PCF de 64 muestras mostrada anteriormente.



Figura 24: Renderizado de sombras suaves: cuatro muestras por píxel, tramadas.

Programa de sombras suaves/PCF

Como se mencionó anteriormente, el cálculo de las sombras suaves se puede realizar completamente en el sombreador de fragmentos. El programa 8.2 muestra el sombreador de fragmentos que reemplaza al de la figura 7. Se resaltan las adiciones al PCF.

Programa 8.2 Filtrado de Porcentaje Más Cercano (PCF)

El sombreador de fragmentos que se muestra en el programa 8.2 contiene código para las sombras suaves PCF de cuatro y 64 muestras. Primero, se define la función `lookup()` para simplificar el proceso de muestreo. Esta función realiza una llamada a la función `GLSL textureProj()`, que realiza una búsqueda en la textura de la sombra, pero se compensa con una cantidad especificada (`ox,oy`). El desplazamiento se multiplica por $1/\text{window size}$, que aquí simplemente hemos codificado a 0.001, asumiendo un tamaño de ventana de 1000×1000 píxeles⁴.

El cálculo de tramado de cuatro muestras aparece resaltado en `main()` y sigue el algoritmo descrito en la sección anterior. Se ha añadido un factor de escala `swidth` que permite ajustar el tamaño de la región “suave” en el borde de las sombras.

A continuación, se muestra el código de 64 muestras, que está comentado. Se puede utilizar en lugar del cálculo de cuatro muestras descomentándolo y comentando el código de cuatro muestras. El factor de escala `swidth` del código de 64 muestras se utiliza como tamaño de paso en el ciclo anidado que muestrea puntos a distintas distancias del píxel que se está renderizando.

Por ejemplo, utilizando el valor de ancho mostrado (2.5), se muestrearían los puntos a lo largo de cada eje a distancias de 1.25, 3.75, 6.25 y 8.25 en ambas direcciones. Luego, se escalarían según el tamaño de la ventana (como se describió anteriormente) y se utilizarían como coordenadas de textura en la textura de la sombra. Con este número de muestras, generalmente no es necesario el dithering para obtener buenos resultados.

La figura 25 muestra nuestro ejemplo de mapeo de sombras de torus/pirámide en ejecución, que incorpora sombreado suave PCF con el sombreador de fragmentos del Programa 8.2, tanto para enfoques de cuatro muestras como de 64 muestras. El valor de ancho elegido depende de la escena; para el ejemplo de toro/pirámide se estableció en 2.5, mientras que para el ejemplo del delfín mostrado previamente en la figura 21, se estableció en 8.0.

Notas adicionales

En este documento, solo hemos presentado las introducciones más básicas al mundo de las sombras en gráficos 3D. Incluso el uso de los métodos básicos de mapeo de sombras presentados aquí probablemente requerirá un estudio más profundo si se utilizan en escenas más complejas.

⁴ También hemos multiplicado el desplazamiento por el componente `w` de la coordenada de sombra, ya que OpenGL divide automáticamente la coordenada de entrada por `w` durante la búsqueda de texturas. Esta operación, denominada división perspectiva, la habíamos omitido hasta ahora, pero es necesario tenerla en cuenta aquí. Para más información sobre la división perspectiva, consulte la respectiva documentación.

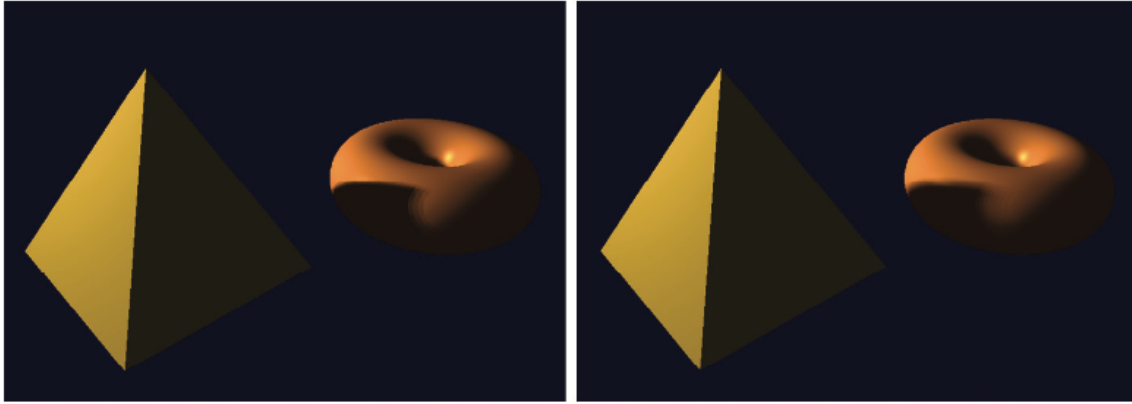


Figura 25: Renderizado de sombras suaves PCF: 4 muestras por píxel, con tramado (izquierda) y 64 muestras por píxel, sin tramado (derecha).

Por ejemplo, al añadir sombras a una escena donde algunos objetos están texturizados, es necesario asegurarse de que el sombreador de fragmentos distinga correctamente entre la textura de la sombra y otras texturas. Una forma sencilla de hacerlo es vincularlos a diferentes unidades de textura, como:

```
layout (binding = 0) uniform sampler2DShadow shTex;  
layout (binding = 1) uniform sampler2D otherTexture;
```

Entonces, la aplicación C++/OpenGL puede hacer referencia a los dos samplers mediante sus valores de vinculación. Cuando una escena utiliza múltiples luces, se necesitan múltiples texturas de sombra: una para cada fuente de luz. Además, se deberá realizar una primera pasada para cada una, y los resultados se fusionarán en la segunda.

Aunque hemos utilizado la proyección en perspectiva en cada fase del mapeo de sombras, cabe destacar que la proyección ortográfica suele preferirse cuando la fuente de luz es distante y direccional, en lugar de la luz posicional que utilizamos.

La generación de sombras realistas es un área rica y compleja de los gráficos por computadora, y muchas de las técnicas disponibles quedan fuera del alcance de este texto. Se anima a los lectores interesados en más detalles a investigar recursos más especializados.

El segundo fragment shader del Programa 8.2 contiene un ejemplo de una función GLSL, `lookup()`. Al igual que en el lenguaje C, las funciones deben definirse antes (o “encima”) de donde se llaman; de lo contrario, se debe proporcionar una declaración adelantada. En el ejemplo, no se requiere una declaración adelantada porque la función se ha definido antes de su llamada.